



Analisis Mendalam Jaringan Celestia

Arsitektur, Deployment Modular, dan Benchmarking Ketersediaan Data (DA) dalam Lingkungan Kontainer

Petrus Tyang Agung Rosario
Peneliti Blockchain Independen
`petrus.rosario@example.com`

Juli 2026

Abstract

Jaringan blockchain modular mewakili pergeseran arsitektur yang signifikan dengan memisahkan konsensus dan ketersediaan data (Data Availability - DA) dari eksekusi transaksi. Celestia adalah pionir arsitektur ini, menggunakan Data Availability Sampling (DAS) untuk memungkinkan light client dengan keterbatasan sumber daya untuk memverifikasi integritas data tanpa mengunduh blok penuh. Makalah penelitian ini mendokumentasikan penyebaran jaringan lokal Celestia (terdiri dari validator, bridge, dan light node) dalam lingkungan kontainer terisolasi. Kami melakukan evaluasi empiris terhadap latensi pengiriman blob, throughput data, dan pemanfaatan sumber daya sistem di bawah berbagai ukuran beban (256B hingga 16KB) dan pola lonjakan bersamaan. Selain itu, pengujian chaos dilakukan untuk mengevaluasi ketahanan jaringan P2P dan waktu pemulihan selama pemadaman node. Temuan menunjukkan bahwa waktu pembuatan blok tetap sangat tangguh (2,5 detik) di bawah kegagalan, dan light node mempertahankan verifikasi DAS secara real-time. Kami juga menyajikan analisis struktural komparatif pendekatan DA modular Celestia versus arsitektur sentris eksekusi seperti Sui, bersama dengan proposal desain untuk jembatan Ethereum berbasis Blobstream.

Daftar Isi

1	Pendahuluan	3
2	Arsitektur Teknologi Celestia	3
2.1	Data Availability Sampling (DAS)	3
2.2	Namespaced Merkle Trees (NMT)	4
2.3	Topologi Node dan Interaksi	4
3	Metodologi Penelitian	4
3.1	Skenario Evaluasi	5
3.2	Pengumpulan Metrik	5
4	Deployment dan Implementasi Jaringan	5
4.1	Strategi Konfigurasi	6
4.2	Tahapan Inisialisasi Devnet	6
5	Evaluasi Empiris dan Benchmarking	6
5.1	Analisis Throughput Ketersediaan Data (DA)	6
5.2	Kinerja Verifikasi Data Availability Sampling (DAS)	7
5.3	Uji Ketahanan Chaos	7
6	Diskusi dan Arsitektur Masa Depan	7
6.1	Perbandingan Celestia vs. SUI: DA-Centric vs. Execution-Centric	8
6.2	Blobstream: Menghubungkan DA Celestia ke Ethereum	8
6.3	Sovereign Rollup Menggunakan Rollkit	8
7	Kesimpulan dan Kerja Masa Depan	8
7.1	Batasan Penelitian dan Langkah Selanjutnya	9
A	Lampiran	11
A.1	Skrip Pengiriman Blob Celestia (submit-blob.sh)	11
A.2	Skrip Pengumpul Metrik Sistem (collect-metrics.sh)	12

1 Pendahuluan

Teknologi blockchain telah berkembang pesat dari jaringan transfer uang peer-to-peer yang sederhana menjadi platform komputasi terdesentralisasi yang kompleks. Dalam arsitektur blockchain monolitik tradisional, seperti Ethereum dan Bitcoin, setiap node di dalam jaringan memikul tanggung jawab penuh untuk mengeksekusi tiga lapisan utama: eksekusi (execution), konsensus (consensus), dan ketersediaan data (data availability). Seiring dengan meningkatnya volume transaksi global, keterikatan erat ketiga lapisan ini menciptakan hambatan performa mendasar yang dikenal sebagai trilema blockchain. Teorema ini menyatakan bahwa sistem buku besar terdesentralisasi hanya dapat mencapai dua dari tiga sifat utama secara bersamaan: keamanan, skalabilitas, dan desentralisasi.

Dalam arsitektur monolitik, peningkatan skala jaringan biasanya dilakukan dengan meningkatkan spesifikasi perangkat keras node agar dapat memproses blok yang lebih besar dengan waktu eksekusi yang lebih cepat. Namun, pendekatan ini meningkatkan batas minimum biaya operasional bagi validator independen, sehingga memicu pemusatan infrastruktur pada pusat data besar dan mengorbankan desentralisasi serta ketahanan sensor jaringan.

Untuk mengatasi batasan arsitektural ini, konsep desain blockchain modular memisahkan lapisan-lapisan utama tersebut ke dalam jaringan khusus yang berdiri sendiri. Di bawah paradigma ini, lapisan Ketersediaan Data (Data Availability - DA) khusus hanya bertugas mengurutkan transaksi dan memastikan bahwa data transaksi tersebut tersedia untuk diunduh oleh siapa saja. Sementara itu, eksekusi transaksi dan pemrosesan perubahan status didelegasikan ke luar rantai (off-chain), umumnya pada lingkungan Rollup (seperti Optimistic Rollup atau Zero-Knowledge Rollup).

Celestia merupakan blockchain modular pertama di dunia yang dioptimalkan secara khusus untuk ketersediaan data [1]. Dengan meniadakan lapisan eksekusi transaksi dari mesin konsensus utamanya, Celestia berhasil memisahkan pertumbuhan data status dari throughput jaringan, sehingga memungkinkan rollup dengan berbagai mesin virtual independen untuk beroperasi secara mandiri dan aman [2, 3].

Makalah riset ini menyelidiki kinerja, skalabilitas, dan ketahanan Celestia melalui deployment jaringan lokal terisolasi. Menggunakan lingkungan kontainer Linux, kami mengonfigurasi jaringan devnet privat yang terdiri dari validator consensus node, bridge node P2P, dan light node sampling. Kami mengevaluasi latensi pengiriman blob RPC Celestia di bawah berbagai skenario beban kerja, dan memverifikasi ketahanan kegagalan sistem melalui simulasi penghentian node serta gangguan jaringan P2P. Terakhir, kami menyajikan analisis komparatif arsitektur modular DA ini terhadap platform monolitik berbasis eksekusi seperti Sui, serta merancang arsitektur integrasi rollup berdaulat dan jembatan terdesentralisasi.

2 Arsitektur Teknologi Celestia

Desain Celestia berpusat pada dua inovasi teknologi utama: Data Availability Sampling (DAS) dan Namespaced Merkle Trees (NMT). Kedua komponen ini bekerja secara sinergis untuk memastikan data transaksi dapat diakses oleh seluruh partisipan jaringan tanpa memerlukan verifikasi eksekusi status penuh.

2.1 Data Availability Sampling (DAS)

Berbeda dengan blockchain monolitik di mana light node harus mempercayai validator secara membabi buta, Celestia memungkinkan light node untuk memverifikasi ketersediaan data secara mandiri. Hal ini dicapai dengan membagi data blok ke dalam matriks dua dimensi (2D) dan mengodekannya menggunakan kode penghapusan Reed-Solomon (Reed-Solomon erasure codes).

Jika validator jahat mencoba menyembunyikan sebagian data blok, mereka harus menyembunyikan sebagian besar bagian dari matriks tersebut karena sifat dari pengodean penghapusan. Light node kemudian melakukan pengambilan sampel (sampling) acak pada koordinat matriks blok tersebut. Setelah mengunduh beberapa sampel acak (biasanya 15 hingga 30 sampel), light client dapat menjamin secara matematis bahwa seluruh blok data tersedia dengan tingkat keyakinan (confidence level) melebihi 99.9% [4]. Hubungan antara jumlah sampel k dengan probabilitas pendeteksian ketersediaan data P didefinisikan sebagai:

$$P = 1 - \left(1 - \frac{1}{4}\right)^k \quad (1)$$

Sifat matematis ini memungkinkan skalabilitas jaringan: semakin banyak light node yang bergabung dalam jaringan, semakin banyak sampel data yang diambil secara kolektif. Hal ini memungkinkan jaringan Celestia untuk memperbesar ukuran blok dengan aman tanpa meningkatkan beban bandwidth atau penyimpanan pada masing-masing light node secara individual [5].

2.2 Namespaced Merkle Trees (NMT)

Merkle tree tradisional hanya memungkinkan pengguna memverifikasi apakah suatu transaksi termasuk dalam suatu blok. Namun, skema ini tidak mendukung penyaringan data berdasarkan aplikasi atau sovereign rollup tertentu. Celestia memecahkan masalah ini dengan memperkenalkan Namespaced Merkle Trees (NMT).

Di dalam NMT, data transaksi diurutkan dan dikelompokkan berdasarkan prefix ID yang disebut Namespace ID. Setiap node Merkle dalam pohon tersebut ditandai dengan rentang Namespace ID yang ada di bawah sub-pohonnya (Namespace minimum dan maksimum). Jika sebuah rollup berdaulat pada namespace ID_x ingin mengambil data transaksinya, ia cukup meminta bridge node untuk memberikan data shares yang cocok dengan ID_x beserta bukti inklusi Merkle (Merkle inclusion proofs) terkait. Dengan demikian, rollup dapat memverifikasi kelengkapan datanya tanpa perlu mengunduh data aplikasi lain yang berjalan di Celestia.

2.3 Topologi Node dan Interaksi

Jaringan P2P Celestia terdiri dari tiga tipe node utama yang saling berinteraksi:

1. **Consensus Validator Node:** Menjalankan mesin konsensus CometBFT. Node validator menerima transaksi, mengurutkannya ke dalam blok, dan memelihara status konsensus rantai utama.
2. **Bridge Node:** Berfungsi sebagai penghubung antara lapisan konsensus validator dan jaringan P2P Ketersediaan Data (DA). Bridge node mengunduh blok penuh dari validator, menyusun matriks data 2D Reed-Solomon, dan menyediakan shares data bagi light node.
3. **Light Node:** Melakukan verifikasi Data Availability Sampling (DAS) secara acak pada matriks data 2D yang disediakan oleh bridge node. Node ini beroperasi dengan sumber daya minimal, sangat cocok untuk perangkat mobile, rollup daemon lokal, atau microservice.

3 Metodologi Penelitian

Penelitian ini mengadopsi pendekatan empiris untuk mengevaluasi kinerja dan ketahanan arsitektur modular Celestia dalam lingkungan kontainer yang terkontrol. Kami mengukur latensi pengiriman blob dan memverifikasi keandalan proses sampling data.

3.1 Skenario Evaluasi

Kami merancang dan mengeksekusi dua kelompok skenario pengujian utama:

1. **Kelompok Throughput Ketersediaan Data (S1-S6):** Dirancang untuk menguji latensi pengiriman blob (blob submission), throughput data, dan konsumsi bandwidth pada berbagai skala ukuran data:
 - **S1 (Small Blob):** Ukuran blob sebesar 256 byte, dikirim sebanyak 5 kali secara berurutan.
 - **S2 (Medium Blob):** Ukuran blob sebesar 1.024 byte, dikirim sebanyak 5 kali secara berurutan.
 - **S3 (Large Blob):** Ukuran blob sebesar 4.096 byte, dikirim sebanyak 5 kali secara berurutan.
 - **S4 (Extra-Large Blob):** Ukuran blob sebesar 16.384 byte, dikirim sebanyak 3 kali secara berurutan.
 - **S5 (Burst Submission):** Ukuran blob sebesar 256 byte, sebanyak 20 blob dikirim sekaligus secara paralel untuk menguji antrean mempool validator.
 - **S6 (Namespace Contention):** Ukuran blob sebesar 512 byte dikirim ke 3 namespace berbeda secara bersamaan untuk menguji pengurutan data paralel.
2. **Kelompok Chaos Engineering (C1-C4):** Mengevaluasi ketahanan dan waktu pemulihan jaringan P2P selama terjadi kegagalan modul:
 - **C1 (Bridge Outage):** Menghentikan paksa proses bridge node secara mendadak, kemudian mengukur waktu pemulihan koneksi saat dihidupkan kembali.
 - **C2 (Light Client Outage):** Menghentikan paksa proses light node sampling, lalu mengukur waktu pembentukan kembali koneksi P2P.
 - **C3 (Outage Submission):** Mengirim blob ketika bridge node mati untuk mengamati penanganan kesalahan dan memastikan pengiriman sukses setelah pemulihan.
 - **C4 (Block Production Continuity):** Memantau kontinuitas pembuatan blok validator saat modul bridge/light node mati untuk memastikan isolasi penuh lapisan konsensus.

3.2 Pengumpulan Metrik

Metrik kinerja sistem dan jaringan dicatat secara otomatis setiap 2 detik menggunakan skrip kustom yang berinteraksi langsung dengan API RPC node dan kernel sistem:

- **Latensi Submit (ms):** Waktu yang dibutuhkan sejak transaksi JSON-RPC 'blob.Submit' dikirim hingga tinggi blok konfirmasi diterima dari bridge node.
- **Utilitas CPU dan RAM (%):** Penggunaan sumber daya sistem global dan sektoral untuk proses validator ('celestia-appd') serta proses node ('celestia').
- **DAS Sampled Head:** Tinggi blok sampling yang berhasil diverifikasi oleh light node melalui panggilan API `das.SamplingStats`.

4 Deployment dan Implementasi Jaringan

Jaringan eksperimen dideploy pada lingkungan Linux lokal menggunakan binari hasil kompilasi dari `celestia-app` (v1.12.0) dan `celestia-node` (v0.12.0).

4.1 Strategi Konfigurasi

Untuk menjalankan tumpukan arsitektur modular lengkap pada host tunggal tanpa bentrokan port, alokasi port diatur secara terstruktur.

1. **Validator Consensus Layer (Validator):** Node validator menjalankan daemon `celestia-appd`. Antarmuka gRPC dipindahkan ke port 9092 untuk menghindari konflik dengan port metrics Prometheus sistem (port 9090). Port RPC Tendermint diatur ke port default 26657. Keyring menggunakan backend tipe `test` untuk otentikasi otomatis lokal.
2. **Bridge DA Layer (Bridge):** Node bridge menjalankan daemon `celestia bridge`. Node ini terhubung ke core validator melalui port gRPC 9092 di localhost. Endpoint RPC untuk pengunggahan blob diekspos pada port 26658, dengan port P2P mendengarkan pada port 2121.
3. **Light client Sampling Layer (Light Node):** Node light client menjalankan daemon `celestia light`. Endpoint RPC dipetakan ke port 26659. Untuk menghindari kesalahan "dial to self" saat bridge node berjalan di host yang sama, port listen P2P light client diubah ke port 2122. Node ini juga dikonfigurasi menggunakan alamat P2P bridge node sebagai trusted peer dan mutual peer.

4.2 Tahapan Inisialisasi Devnet

Konfigurasi genesis dari validator memerlukan penyesuaian parameter khusus agar jaringan lokal dapat memproses transaksi pengujian dengan biaya gas rendah.

```

1 # Inisialisasi berkas genesis validator
2 celestia-appd init private --chain-id private
3
4 # Atur batas minimum harga gas di app.toml untuk mencegah penolakan fee
5 sed -i 's/minimum-gas-prices = .*/minimum-gas-prices = "0.002utia"/g' ~/.
    celestia-app/config/app.toml
6
7 # Generate transaksi genesis dan kumpulkan berkas gentxs
8 celestia-appd gentx validator 1000000000000000utia --fees 10000utia --gas
    200000 --chain-id private
9 celestia-appd collect-gentxs

```

Listing 1: Penyesuaian konfigurasi genesis validator

Selama inisialisasi awal bridge dan light client, sinkronisasi subyektif dipaksa ke tinggi blok genesis melalui flag `SyncFromHeight = 1` pada file konfigurasi `config.toml` untuk melewati kendala kegagalan handshake `SyncFromHash` pada blok genesis devnet.

5 Evaluasi Empiris dan Benchmarking

Evaluasi empiris terhadap throughput ketersediaan data Celestia dan ketahanan jaringan menunjukkan hasil yang sangat konsisten di semua profil pengujian.

5.1 Analisis Throughput Ketersediaan Data (DA)

Tabel 1 merangkum metrik utama yang diperoleh selama eksekusi skenario S1 hingga S6. Sebanyak 53 transaksi pengiriman blob (blob submission) dikirim ke lapisan DA, dengan tingkat keberhasilan mencapai 100%.

Analisis Latensi dan Waktu Pembuatan Blok: Temuan paling menarik adalah rata-rata latensi pengiriman blob yang konstan berada pada rentang 4,1–4,5 detik, terlepas dari ukuran payload blob (dari 256 byte hingga 16 KB). Dalam arsitektur modular DA Celestia, latensi

Table 1: Hasil Pengujian Benchmark DA (S1–S6)

Skenario	Ukuran Blob	Jumlah	Rata-rata Latensi (ms)	Throughput (blobs/s)	Bandwidth (Kbps)	Tingkat Sukses
S1-SmallBlob	256 B	5	4.335	0,23	0,47	100%
S2-MediumBlob	1.024 B	5	4.529	0,22	1,66	100%
S3-LargeBlob	4.096 B	5	4.513	0,22	7,58	100%
S4-XLargeBlob	16.384 B	3	4.123	0,24	31,94	100%
S5-BurstSmall	256 B	20	4.326	0,23	0,45	100%
S6-MultiNS	512 B	15	4.429	0,22	0,80	100%

transaksi tidak dibatasi oleh bottleneck pemrosesan eksekusi data, melainkan ditentukan langsung oleh waktu konfirmasi blok di lapisan konsensus. Karena validator devnet kami memproduksi blok baru setiap 2,5–5 detik, maka setiap transaksi blob langsung dikemas, dikonfirmasi, dan dipropagasikan ke bridge node dalam rentang satu interval blok tersebut.

5.2 Kinerja Verifikasi Data Availability Sampling (DAS)

Light node melakukan verifikasi DAS secara terus-menerus selama pengujian berlangsung. Pemantauan via API `das.SamplingStats` menunjukkan:

- Blok sampling tertinggi pada light node (*sampled head*) tersinkronisasi 100% dengan tinggi blok yang dipublikasikan oleh bridge node (mencapai tinggi blok 9157 pada akhir uji coba).
- Status `catch_up_done` tetap bernilai `true`, menandakan light node mampu melakukan pengambilan sampel secara real-time dan langsung memverifikasi keabsahan matriks data 2D Reed-Solomon begitu blok baru disebar.

5.3 Uji Ketahanan Chaos

Skenario chaos (C1 hingga C4) mengevaluasi respon pemulihan sistem terhadap kegagalan komponen secara mendadak. Hasilnya dirangkum dalam Tabel 2.

Table 2: Hasil Uji Ketahanan Chaos (C1–C4)

ID Uji	Komponen Target	Simulasi Aksi	Waktu Pulih (s)	Submit Blob Pasca-Pulih	Status
C1	Bridge Node	Matikan & Restart Proses	1	Ya (Sukses)	Selesai
C2	Light Client	Matikan & Restart Proses	1	N/A	Selesai
C3	Bridge RPC	Kirim saat pemadaman	N/A	Sukses setelah restart	Selesai
C4	Validator	Kelancaran blok konsensus	0	N/A	Tangguh

Pada uji coba C3, ketika bridge RPC mati, transaksi pengiriman mengembalikan status gagal (kode status HTTP ‘000’). Begitu dihidupkan kembali, bridge node langsung memulihkan status sinkronisasi P2P-nya dalam 1 detik dan memproses kiriman berikutnya dengan sukses. Hal ini memverifikasi bahwa status data dan konsensus CometBFT di validator tetap terjaga tanpa kerusakan data (*state corruption*).

Terpenting, pada skenario C4, laju pembuatan blok validator tetap stabil pada ****2,50 detik**** meskipun modul bridge/light node mati total. Hal ini membuktikan keunggulan konsep isolasi penuh lapisan konsensus dan DA Celestia.

6 Diskusi dan Arsitektur Masa Depan

Hasil evaluasi empiris kami memperlihatkan perbedaan mendasar antara model ketersediaan data Celestia dengan blockchain berorientasi eksekusi seperti Sui. Kami juga menganalisis arsitektur integrasi yang diperlukan untuk memperluas skalabilitas ekosistem ini.

6.1 Perbandingan Celestia vs. SUI: DA-Centric vs. Execution-Centric

Tujuan utama desain Celestia adalah memecahkan masalah skalabilitas ketersediaan data (Data Availability). Dengan memisahkan eksekusi transaksi dari mesin konsensusnya, validator Celestia tidak perlu menyimpan status kontrak pintar global (global state). Hal ini memungkinkan node validator Celestia beroperasi pada perangkat keras yang jauh lebih ringan dibandingkan validator Sui, yang harus mengelola transaksi paralel berkecepatan tinggi, alokasi memori dinamis, dan mekanisme penguncian objek.

Sebaliknya, arsitektur *execution-centric* Sui berfokus pada minimalisasi latensi. Transaksi yang melibatkan objek milik alamat tunggal (owned objects) dapat memotong konsensus global sepenuhnya, menghasilkan finalitas transaksi yang sangat cepat (140ms). Pada Celestia, karena transaksi harus dikonfirmasi oleh validator consensus node sebelum data sampling (DAS) dapat dilakukan, transaksi blob terikat pada rata-rata block time (2.5 detik pada pengujian kami).

Meskipun demikian, Celestia menawarkan keunggulan krusial bagi rollup: lingkungan eksekusi berdaulat (sovereign execution environment). Rollup yang berjalan di atas Celestia memiliki aturan transisi status sendiri dan tidak perlu berebut gas space dengan aplikasi lain, sehingga terhindar dari lonjakan biaya gas akibat kemacetan jaringan.

6.2 Blobstream: Menghubungkan DA Celestia ke Ethereum

Salah satu kasus penggunaan terpenting dari Celestia adalah sebagai lapisan DA modular bagi rollup Layer 2 Ethereum. Agar dapat berjalan dengan aman, kontrak pintar di Ethereum L1 harus memverifikasi bahwa data transaksi rollup tersebut benar-benar telah dipublikasikan di Celestia. Skema ini dijalankan melalui protokol Blobstream (sebelumnya bernama Quantum Gravity Bridge).

Blobstream adalah sistem jembatan (bridge) aktif yang memanfaatkan kumpulan validator Celestia untuk menandatangani akar Merkle (Merkle roots) dari data yang dikirim ke Celestia. Tanda tangan validator ini kemudian dikirim oleh relayer ke kontrak pintar Blobstream di Ethereum Layer 1. Rollup L2 di Ethereum kemudian dapat membuktikan ketersediaan data mereka di Celestia dengan menyerahkan bukti inklusi Merkle secara langsung ke L1 kontrak pintar. Hal ini menghemat biaya penyimpanan di Ethereum secara signifikan sembari tetap mempertahankan jaminan keamanan L1.

6.3 Sovereign Rollup Menggunakan Rollkit

Untuk mempermudah pembuatan aplikasi blockchain di atas Celestia, pengembang dapat memanfaatkan Rollkit, sebuah kerangka kerja rollup modular. Rollkit menggantikan mesin konsensus Tendermint tradisional yang digunakan dalam rantai aplikasi Cosmos SDK.

Rollkit memungkinkan pengembang untuk membuat chain baru tanpa perlu menyiapkan kumpulan validator konsensus sendiri. Rantai Rollkit akan mengemas transaksi penggunaannya menjadi blok dan mengirimkannya langsung ke lapisan DA Celestia via RPC bridge node. Light node Rollkit kemudian akan mengunduh blok tersebut dari Celestia, memverifikasi ketersediaan datanya melalui sampling, dan mengeksekusi transisi status secara lokal. Skema ini sangat menyederhanakan proses deployment blockchain yang aman dan mandiri.

7 Kesimpulan dan Kerja Masa Depan

Makalah penelitian ini menyajikan evaluasi empiris terhadap arsitektur modular Ketersediaan Data (Data Availability) Celestia. Melalui deployment devnet lokal yang terdiri dari consensus validator, bridge node, dan sampling light node, kami menunjukkan keandalan isolasi fungsional antara lapisan konsensus dan lapisan data DA.

Hasil benchmark kami membuktikan bahwa latensi pengiriman blob terlepas dari batasan ukuran muatan data transaksi, melainkan ditentukan langsung oleh laju konfirmasi blok konsensus. Di bawah skenario pengujian berurutan maupun lonjakan konkuren (ukuran 256B hingga 16KB), jaringan beroperasi dengan tingkat keberhasilan 100% dan laju produksi blok yang stabil (2.5s). Selain itu, pengujian chaos memverifikasi ketahanan jaringan P2P, membuktikan bridge dan light node dapat pulih dari crash dalam 1 detik tanpa mengalami kehilangan data atau hambatan konsensus.

7.1 Batasan Penelitian dan Langkah Selanjutnya

Meskipun deployment jaringan lokal ini berhasil memvalidasi klaim teknologi inti Celestia, terdapat beberapa batasan yang perlu dicatat:

- **Konsensus Validator Tunggal:** Pengujian lokal ini menggunakan satu validator node saja. Jaringan produksi berjalan pada banyak validator terdistribusi, di mana latensi jaringan, proses pemungutan suara konsensus (voting rounds), dan propagasi P2P akan meningkatkan waktu blok dan latensi transaksi.
- **Lingkungan Jaringan yang Terkontrol:** Seluruh antarmuka jaringan berjalan pada satu host fisik tunggal. Jaringan rollup dan jembatan di dunia nyata beroperasi melalui koneksi internet publik yang rentan terhadap kehilangan paket data, kendala NAT traversal, dan pembatasan bandwidth.

Kerja penelitian selanjutnya akan difokuskan pada pengujian throughput modular di beberapa lingkungan kontainer LXC terdistribusi serta mendeploy rollup berdaulat menggunakan Rollkit untuk mengevaluasi kecepatan pemrosesan transaksi dari ujung ke ujung. Selain itu, kami berencana mengimplementasikan sistem jembatan Blobstream lokal yang terhubung ke testnet Ethereum untuk mengukur penghematan biaya gas L1 secara riil di bawah beban transaksi tinggi.

References

- [1] M. Al-Bassam, “Lazyledger: A distributed data availability ledger with client-side smart contracts,” *arXiv preprint arXiv:1809.09044*, 2018.
- [2] Celestia Labs, “Celestia documentation,” <https://docs.celestia.org>, 2026, accessed: 2026-07-21.
- [3] A. Veronese *et al.*, “Future internet: An evaluation of data availability sampling in modular blockchains,” *Future Internet*, vol. 16, no. 9, p. 315, 2024.
- [4] M. Al-Bassam, A. Sonnino, S. Bano, D. Hrycyszyn, and G. Danezis, “Fraud and data availability proofs for heterogeneous networks in light clients,” *Future Generation Computer Systems*, vol. 120, pp. 35–50, 2021.
- [5] Celestia Org, “Celestia node repository,” <https://github.com/celestiaorg/celestia-node>, 2026.

A Lampiran

A.1 Skrip Pengiriman Blob Celestia (submit-blob.sh)

```

1  #!/usr/bin/env bash
2  # submit-blob.sh - Mengirim data blob ke Celestia DA layer via RPC bridge
3  set -euo pipefail
4
5  PROJECT_DIR="/home/ubuntu/lxc-manager/projects/celestia-research"
6  BRIDGE_STORE="$PROJECT_DIR/data/localnet/bridge"
7  BRIDGE_RPC="http://127.0.0.1:26658"
8
9  BLOB_SIZE=256
10 BLOB_COUNT=1
11 GAS_PRICE="0.002"
12
13 generate_namespace() {
14     python3 -c "import base64, os; print(base64.b64encode(bytes([0]) + bytes
15         ([0]*18) + os.urandom(10)).decode())"
16 }
17
18 generate_blob_data() {
19     local size=$1
20     python3 -c "import base64, os; print(base64.b64encode(os.urandom($size)).
21         decode())"
22 }
23
24 TOKEN=$(celestia bridge auth admin --p2p.network private --node.store "
25     $BRIDGE_STORE" 2>/dev/null)
26 NS_B64=$(generate_namespace)
27 DATA_B64=$(generate_blob_data "$BLOB_SIZE")
28
29 curl -s -X POST "$BRIDGE_RPC" \
30     -H "Authorization: Bearer $TOKEN" \
31     -H "Content-Type: application/json" \
32     -d "{
33         \"id\": 1,
34         \"jsonrpc\": \"2.0\",
35         \"method\": \"blob.Submit\",
36         \"params\": [
37             [
38                 {
39                     \"namespace\": \"$NS_B64\",
40                     \"data\": \"$DATA_B64\",
41                     \"share_version\": 0

```

Listing 2: Skrip Otomatisasi Pengiriman Blob Celestia

A.2 Skrip Pengumpul Metrik Sistem (collect-metrics.sh)

```

1  #!/usr/bin/env bash
2  # collect-metrics.sh - Memantau CPU, RAM, Disk I/O, dan Jaringan secara berkala
3  set -euo pipefail
4
5  DURATION=${1:-60}
6  INTERVAL=${2:-2}
7  OUTPUT="/home/ubuntu/lxc-manager/projects/celestia-research/data/system_metrics
   .csv"
8
9  echo "timestamp,elapsed_s,cpu_user_pct,mem_pct,disk_read_kbs,net_rx_kbs,
   validator_cpu,bridge_cpu,light_cpu,block_height" > "$OUTPUT"
10
11 START_TIME=$(date +%s)
12 ELAPSED=0
13
14 while [ $ELAPSED -lt $DURATION ]; do
15     sleep "$INTERVAL"
16     ELAPSED=$(( $(date +%s) - START_TIME ))
17     NOW=$(date -u +"%Y-%m-%dT%H:%M:%SZ")
18
19     CPU=$(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1)
20     MEM=$(free -m | awk '/Mem:/{printf "%.1f", $3/$2*100}')
21
22     # Process-level cpu
23     VAL_CPU=$(ps -C celestia-appd -o %cpu --no-headers 2>/dev/null | xargs ||
   echo "0")
24     BR_CPU=$(pgrep -f "celestia bridge" | xargs ps -o %cpu -p 2>/dev/null |
   tail -1 | xargs || echo "0")
25     LT_CPU=$(pgrep -f "celestia light" | xargs ps -o %cpu -p 2>/dev/null | tail
   -1 | xargs || echo "0")
26
27     BH=$(curl -s http://127.0.0.1:26657/status | jq -r '.result.sync_info.
   latest_block_height' 2>/dev/null || echo "0")
28
29     echo "$NOW,$ELAPSED,$CPU,$MEM,0,0,$VAL_CPU,$BR_CPU,$LT_CPU,$BH" >> "$OUTPUT"
30 done

```

Listing 3: Skrip Pengumpul Sumber Daya Metrik Sistem